

# Specula Walkthrough

---

2026-09-26

1. The problem
2. Glossary
3. The pipeline at a glance
4. Stage 1: Direction ( `profile.yaml` )
5. Stage 2: Collection ( `cti/collect.py` )
6. Stage 3: Processing ( `cti/process.py` )
7. Stage 4: Analysis ( `cti/analyse.py` )
8. Stage 5: Dissemination ( `cti/disseminate.py` )
9. Stage 6: Feedback ( `exceptions.csv` )
10. The Specula console ( `app.py` )
11. Security of the pipeline itself
12. Known limitations
13. Quick reference

*Specula* (Latin: a watchtower) is a small, explainable Cyber Threat Intelligence (CTI) pipeline. It takes **every published CVE in the world** and narrows it down to the few that matter to **one organisation**, and it writes down the reason for every decision.

This walkthrough explains how it works, one CTI lifecycle stage at a time. It is written for security analysts, IT managers and students. You can read it on its own, or next to the Specula console ( `python app.py` ), which has one screen per section below. To set Specula up for your own organisation, follow [CUSTOMISE.md](#).

# 1. The problem

---

About 40,000 CVEs are published every year, and hundreds of thousands are on record. Most vulnerability programmes rank them by **CVSS**, a 0–10 severity score. CVSS measures how bad a flaw *could* be. It says nothing about:

- whether the organisation **runs** the affected software,
- whether a **supplier** runs it on the organisation's behalf,
- whether anyone is **actually exploiting** it, and whether that someone is interested in *this* organisation.

Research behind EPSS shows that only a small share of CVEs is ever exploited. Patching by CVSS alone means spending most of the effort on flaws nobody uses. Specula replaces the question "*how severe is it?*" with "***is it being used, by whom, against something we own or depend on?***". That shift is intelligence-led vulnerability management.

## 2. Glossary

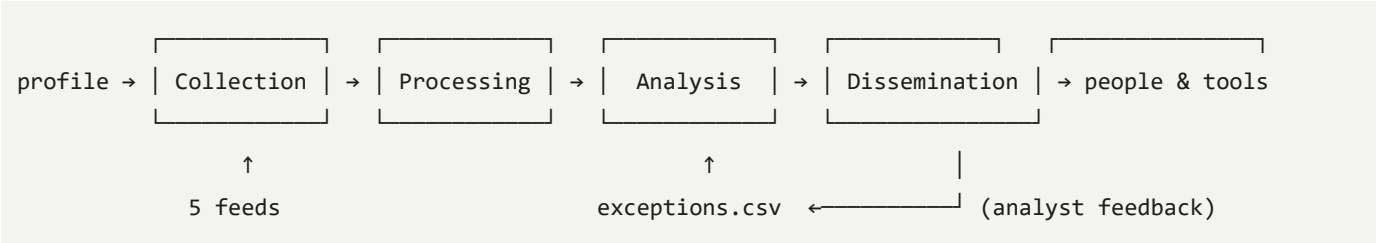
| Term                    | What it is                                                                                                   | Why it matters here                                        |
|-------------------------|--------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| <b>CVE</b>              | A unique ID for a publicly known vulnerability, e.g. CVE-2021-26855                                          | Everything is keyed on it                                  |
| <b>NVD</b>              | The US National Vulnerability Database. It adds severity, weakness type and affected products to every CVE   | The source of <i>all</i> CVEs                              |
| <b>CPE</b>              | A machine-readable product name, e.g. cpe:2.3:a:microsoft:exchange_server                                    | How a CVE is matched to software you run                   |
| <b>CVSS</b>             | A 0–10 severity score                                                                                        | Impact, not likelihood                                     |
| <b>KEV</b>              | CISA's <i>Known Exploited Vulnerabilities</i> catalogue                                                      | Proof of exploitation in the wild, the strongest signal    |
| <b>EPSS</b>             | FIRST's <i>Exploit Prediction Scoring System</i> : the probability (0–1) of exploitation in the next 30 days | Predicts what will be used soon                            |
| <b>MITRE ATT&amp;CK</b> | A catalogue of adversary groups and the techniques they use                                                  | Links a CVE to <i>who</i> uses it and <i>what for</i>      |
| <b>CTID mappings</b>    | The Center for Threat-Informed Defense's mapping from KEV CVEs to ATT&CK techniques                          | The bridge between a CVE and attacker behaviour            |
| <b>SSVC</b>             | A decision model that sorts vulnerabilities into Act / Attend / Track / Ignore                               | Turns scores into actions                                  |
| <b>STIX 2.1</b>         | The standard format for sharing threat intelligence                                                          | Lets results flow into MISP, OpenCTI or Microsoft Sentinel |
| <b>TLP</b>              | Traffic Light Protocol sharing labels (CLEAR / GREEN / AMBER / RED)                                          | Marks who may see the output                               |
| <b>PIR</b>              | Priority Intelligence Requirement: a question leadership needs answered                                      | Gives the whole pipeline its purpose                       |

### 3. The pipeline at a glance

Specula follows the classic CTI lifecycle. Each stage is one file, so the code reads like the process:

| Stage            | File               | Input → Output                                                |
|------------------|--------------------|---------------------------------------------------------------|
| 1. Direction     | profile.yaml       | Human judgement → organisation, PIRs, threat groups, weights  |
| 2. Collection    | cti/collect.py     | Five public feeds → local SQLite database <code>cve.db</code> |
| 3. Processing    | cti/process.py     | Your assets + suppliers × all CVEs → findings                 |
| 4. Analysis      | cti/analyse.py     | Findings × threat profile → risk score + decision tier        |
| 5. Dissemination | cti/disseminate.py | Ranked findings → dashboard, report, brief, CSV, STIX         |
| 6. Feedback      | exceptions.csv     | Analyst decisions → suppressed or returning findings          |

`run.py` runs all six in order, which is what the daily scheduler calls. `app.py` is the console, which runs the same code one stage at a time and shows you each step.



## 4. Stage 1: Direction ( `profile.yaml` )

---

**What happens:** a person states who the organisation is, what it needs to know, and who is likely to attack it. Nothing is automated here, on purpose: choosing adversaries is a judgement call.

### Contents:

- **Organisation:** name, sector, country.
- **PIRs:** for example, "Which actively exploited vulnerabilities affect our internet-facing systems?"
- **Threat groups:** MITRE ATT&CK IDs of the adversaries relevant to the sector and region, chosen from national CERT and sector ISAC reporting.
- **Weights, exposure, tiers:** every number the pipeline later uses to decide.

**Why it matters:** the same CVE is more urgent for a hospital targeted by a ransomware group that exploits it than for a firm nobody targets with it. Direction is what makes the output *threat-informed* instead of generic.

**Try it:** in the console, open **Direction**, change `threat_profile: 0.2` to `0.5`, save, then re-rank in **Analysis** and watch CVEs used by your profiled groups move up.

## 5. Stage 2: Collection ( `cti/collect.py` )

**What happens:** five public feeds are downloaded into `cve.db`, a single SQLite file.

| Feed              | Publisher | What it adds                                                                                             | Refresh                           |
|-------------------|-----------|----------------------------------------------------------------------------------------------------------|-----------------------------------|
| NVD API 2.0       | NIST      | Every CVE: CVSS, CWE, affected products (CPE + version ranges)                                           | Incremental by last-modified date |
| KEV               | CISA      | Known-exploited CVEs, including the ransomware flag                                                      | Full, daily                       |
| EPSS              | FIRST     | Exploitation probability for every CVE                                                                   | Full, daily                       |
| ATT&CK Enterprise | MITRE     | Techniques, adversary groups, which group uses which technique, CVEs named in group and campaign reports | Full                              |
| KEV → ATT&CK      | CTID      | Which techniques each KEV CVE enables                                                                    | Newest release                    |

### How it works:

- The first NVD run downloads the full history (about 400,000 CVEs). Later runs ask only for records changed since the last run, respecting NVD's rate limits and its 120-day window limit.
- **Poisoning guard:** a feed that comes back suspiciously small (for example, fewer than 1,000 KEV entries) is rejected, and the previous data is kept. A broken or tampered download can't wipe the database.
- All downloads use HTTPS with certificate verification. The optional NVD API key comes from the operating system's credential store, never from a file.

**Why it matters:** collection is automatic and needs no people. Everything after this point runs offline on local data, so results are reproducible.

**Try it: Collection → Quick sync since a date** pulls only recent changes and shows the progress in the console's terminal panel.

## 6. Stage 3: Processing ( `cti/process.py` )

---

**What happens:** the global feed meets the organisation. Every asset and every supplier is matched against every CVE.

**Owned assets** ( `assets.csv` ): each row has a CPE prefix and a version. A CVE matches when one of its NVD product entries starts with that prefix **and** the version falls inside the entry's range ( `versionStartIncluding` , `versionEndExcluding` and so on). Versions compare numerically, so `10.0.17763.5329` > `10.0.17763.999` . With no version given, every CVE for the product counts, and the report says so.

**Suppliers** ( `third_parties.csv` ): you can't see a supplier's patch level. So for suppliers with an unknown version, only CVEs **published or added to KEV in the last 90 days** count. That answers the question *"is my supplier under attack right now?"*.

**Microsoft Defender (optional):** when enabled, the live software inventory replaces hand-written rows, and Defender's own per-device vulnerability list decides which CVEs apply at each patch level.

**Why it matters:** this is the biggest filter. Hundreds of thousands of CVEs become the few thousand that touch the estate.

**Try it: Processing** → **CPE lookup:** type `exchange` to see which NVD product names exist and how many CVEs each has.

## 7. Stage 4: Analysis ( `cti/analyse.py` )

**What happens:** each finding gets an explainable risk score and a decision tier.

```
likelihood = 0.4·EPSS + 0.3·inKEV + 0.1·ransomware + 0.2·threat_overlap
impact     = CVSS/10 · weight · exposure
risk       = likelihood · impact
```

| Term           | Meaning                                                                                                                                                 |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| threat_overlap | <b>1.0</b> if a profiled group is known (per ATT&CK) to exploit this CVE; <b>0.5</b> if the CVE enables ATT&CK techniques a profiled group uses; else 0 |
| weight         | Owned asset: criticality 1–3. Supplier: <i>blast radius</i> = data-access factor × (privileged-access factor)                                           |
| exposure       | 1.5 if internet-facing, 1.0 if internal                                                                                                                 |

All numbers come from `profile.yaml`. Every finding carries a written "**why**" listing each term, for example: "*EPSS 0.97; KEV: exploited in the wild; used by ransomware; exploited by APT28 (threat profile); CVSS 9.8; criticality 3; internet-facing*".

**Decision tiers (SSVC-style)**, checked top to bottom:

| Tier          | Rule                                                                           | Action                        |
|---------------|--------------------------------------------------------------------------------|-------------------------------|
| <b>Act</b>    | Exploited (KEV or profiled group), <b>and</b> internet-facing or most critical | Patch or mitigate within 48 h |
| <b>Attend</b> | Exploited, or EPSS ≥ 0.1                                                       | Next patch cycle              |
| <b>Track</b>  | EPSS ≥ 0.01 or CVSS ≥ 7                                                        | Watch for exploitation        |
| <b>Ignore</b> | Everything else                                                                | No action                     |

**Why it matters:** the scoring is **deterministic**. The same data always produces a byte-identical ranking. No AI takes part in any decision, so an auditor or examiner can check every number.

**Try it: Analysis** → **Re-rank** (no download, a few seconds), then read the "why" column of the top 10.

## 8. Stage 5: Dissemination ( `cti/disseminate.py` )

**What happens:** the ranking goes out in five forms for five audiences, all marked **TLP:AMBER**:

| Output                          | For                     | Contents                                                                                                             |
|---------------------------------|-------------------------|----------------------------------------------------------------------------------------------------------------------|
| <code>out/dashboard.html</code> | Analysts                | Self-contained page: tier cards, search, sort, filter by owned or supplier                                           |
| <code>out/report.md</code>      | Security team           | Decision summary, per-asset table, threat-group hits, ATT&CK tactic view, top findings, supplier section, exceptions |
| <code>out/brief.md</code>       | Management              | One page: headline risk, this week's actions, supplier actions                                                       |
| <code>out/report.csv</code>     | Spreadsheets, ticketing | Every finding with every score term                                                                                  |
| <code>out/bundle.json</code>    | Other CTI platforms     | STIX 2.1: vulnerabilities, affected infrastructure, intrusion sets, attack patterns, relationships                   |

STIX object IDs are **deterministic**: the same finding gets the same ID every run, so re-importing into MISP or OpenCTI updates objects instead of duplicating them.

The management brief uses a fixed template by default. Optionally an LLM (local Ollama, Azure OpenAI in your tenant, or Anthropic) writes it instead. The LLM sees only the Act findings, CVE text is fenced as untrusted input against prompt injection, cloud providers get redacted input, and any failure falls back to the template.

**Try it: Dissemination** → open the dashboard and click the tier cards.

## 9. Stage 6: Feedback ( `exceptions.csv` )

---

**What happens:** analysts record decisions: `patched`, `mitigated`, `accepted`, `false_positive` or `not_affected`, for one asset or for all ( `*` ). Those findings drop out of the next ranking and are listed in the report's exceptions section.

Every exception can have an **expiry date**. When it passes, the finding returns for review and the report flags it. Risk is never silenced forever by accident.

**Why it matters:** without feedback, the same already-handled CVEs stay at the top every day and people stop reading. The feedback loop keeps the output trustworthy.

**Try it:** add an exception for the top CVE, re-rank, and check that it is gone and counted as suppressed.

## 10. The Specula console (`app.py`)

---

`python app.py` opens `http://127.0.0.1:8765`. It has one screen per stage above, live numbers from `cve.db`, a funnel from *all CVEs* to *act within 48 h*, file editors, and buttons that run the real pipeline in the background. A terminal panel streams everything the code prints.

It is safe by design:

| Risk                                 | Control                                                                                                                 |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Other machines use it                | Listens on <code>127.0.0.1</code> only                                                                                  |
| A malicious website drives it (CSRF) | Every API call needs a random per-start token                                                                           |
| DNS rebinding                        | Requests with a foreign <code>Host</code> header are refused                                                            |
| Arbitrary commands                   | Only five fixed actions exist; inputs are validated                                                                     |
| Arbitrary file edits                 | Only the four configuration files; each is validated before saving, with the previous version kept as <code>.bak</code> |
| Script injection from feed text      | The page inserts text only, never HTML                                                                                  |

`python app.py --pdf` renders this walkthrough to `WALKTHROUGH.pdf` (needs pandoc and Edge or Chrome).

## 11. Security of the pipeline itself

---

Specula holds a map of which vulnerable software runs where, which makes it a target itself.

| Threat                             | Control                                                                                                                 |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Stolen API keys                    | Keys live in the OS credential store ( <code>keyring</code> ), never in files or git                                    |
| Leaked vulnerability map           | Data files are git-ignored; restrict the folder, encrypt the disk, and mark outputs TLP:AMBER (see CUSTOMISE.md step 8) |
| Poisoned feed                      | Size and shape checks before any write; HTTPS with verification                                                         |
| Malicious dependency               | Only 4 direct dependencies, pinned by hash ( <code>pip install --require-hashes</code> )                                |
| Prompt injection via CVE text      | LLM input is fenced as untrusted, and LLM output is display-only and never acted on                                     |
| XSS via feed text in the dashboard | Data is embedded as escaped JSON and rendered with <code>textContent</code> only                                        |
| Silent failures                    | Every run, including failures, appends one line to <code>run.log</code>                                                 |
| Over-privileged integrations       | The Defender app is read-only and uses certificate authentication, not a secret                                         |

## 12. Known limitations

---

- **Version formats:** vendor-specific schemes (for example `2.8_mr10`) compare as text. The CPE "update" field is ignored, so some already-fixed CVEs may still be listed. This errs towards over-reporting, never under-reporting.
- **Assets without versions** are assumed affected by every CVE for that product.
- **Scale:** one database scan per asset. This is fine up to about 100 assets; beyond that, add a CPE index table (the spot is marked in `cti/process.py`).
- **Threat groups** are chosen by people. ATT&CK only knows the CVE usage that has been publicly reported.
- **Single user:** the console is a local tool. Put authentication and HTTPS in front of it before sharing it on a network.

## 13. Quick reference

---

```
python test_pipeline.py      # offline self-check
python app.py                # console at http://127.0.0.1:8765
python app.py --pdf          # WALKTHROUGH.md -> WALKTHROUGH.pdf
python run.py                # full cycle: collect, match, score, report
python run.py --since 2026-01-01 # only CVEs changed since a date
python run.py --skip-collect  # re-rank on existing data
python run.py --suggest-cpe <product> # find NVD product names
```

*Specula by Diljot Singh Johal, released under the MIT License. Feed data belongs to its publishers (NIST NVD, CISA, FIRST, MITRE, the Center for Threat-Informed Defense). Check their terms before redistributing it.*